

**INSTITUTO FEDERAL DE EDUCAÇÃO CIÊNCIA E TECNOLOGIA DE SÃO  
PAULO**

**CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS INTEGRADO AO  
ENSINO MÉDIO**

**SABOR E CLIC**

---

**Documentação do Projeto Integrador do Terceiro Bimestre**

São Paulo

2026

**SUMÁRIO**

- 1 Apresentação do projeto
- 2 Objetivos e escopo
- 3 Concepção do sistema e integração entre disciplinas
- 4 Atores e responsabilidades
- 5 Fluxo geral do produto

|    |                                                 |
|----|-------------------------------------------------|
| 6  | Reserva                                         |
| 7  | Sessão                                          |
| 8  | Cardápio e carrinho                             |
| 9  | Pedidos                                         |
| 10 | KDS                                             |
| 11 | Administração de Bancadas e catálogo            |
| 12 | Arquitetura da aplicação                        |
| 13 | Lógica JavaScript                               |
| 14 | Arquitetura Flask e rotas essenciais            |
| 15 | Banco de dados e DER                            |
| 16 | Segurança e níveis de acesso                    |
| 17 | Identidade visual e prototipação                |
| 18 | Requisitos funcionais e não funcionais          |
| 19 | Tecnologias utilizadas                          |
| 20 | O que concluímos e próximas etapas              |
| 21 | Atendimento aos requisitos do terceiro bimestre |
|    | Referências                                     |

## **1 APRESENTAÇÃO DO PROJETO**

Nós desenvolvemos o Sabor e Clic como uma proposta de sistema para organizar a operação de uma cozinha compartilhada com Bancadas reserváveis e atendimento gastronômico integrado. Nosso ponto de partida foi um problema recorrente: quando Reservas, disponibilidade de espaços, Cardápio e Pedidos são controlados por planilhas ou anotações isoladas, surgem conflitos de horário, falta de rastreabilidade e dificuldade para acompanhar a produção. Por isso, concebemos uma experiência única que liga o Cliente à cozinha e à administração.

Nossa documentação acompanha o fluxo do produto: Reserva → Sessão → Cardápio e Pedidos → KDS. Escolhemos esse encadeamento porque cada Pedido precisa existir dentro de um contexto de atendimento. A Reserva define quem utilizará qual Bancada e em qual período; a Sessão representa o uso efetivo dessa Reserva; o Cardápio permite selecionar itens; o Pedido registra a solicitação; e o KDS oferece à cozinha uma visão organizada da produção.

Ao longo do terceiro bimestre, reunimos contribuições de Artes Visuais e Design, JavaScript, Python, Banco de Dados e Segurança. O front-end evoluiu para uma aplicação multipágina navegável com telas específicas para Cliente, Cozinheiro e Administrador. Também modularizamos a lógica JavaScript e implementamos um back-end Flask simulado, com estrutura inicial da aplicação e endpoints fundamentais. A persistência definitiva e a comunicação real por Fetch/JSON permanecem para o quarto bimestre.

## **2 OBJETIVOS E ESCOPO**

### **2.1 Objetivo geral**

Nosso objetivo é construir um sistema web que permita administrar Bancadas gastronômicas, criar Reservas, iniciar Sessões, consultar o Cardápio, enviar Pedidos e acompanhar seu preparo no KDS. Pretendemos centralizar dados que normalmente estariam dispersos e oferecer uma visão adequada para Cliente, Cozinheiro e Administrador.

### **2.2 Objetivos específicos**

- Representar o fluxo completo do atendimento de forma clara e navegável.
- Modelar as entidades, atributos e relacionamentos em um DER e em um dicionário de dados.
- Aplicar JavaScript moderno para manipular o DOM, manter o carrinho e calcular valores em memória.
- Definir uma API Flask organizada por responsabilidades, com comunicação por JSON.
- Planejar autenticação, validação e RBAC para proteger ações e dados.
- Criar identidade visual coerente e interfaces para Cardápio Digital e KDS.

### 2.3 Limites desta etapa

Nesta etapa concluímos o protótipo navegável, a documentação conceitual de dados, as lógicas JavaScript exigidas no terceiro bimestre e uma estrutura inicial de back-end Flask com endpoints simulados. A aplicação ainda não persiste os dados do fluxo completo nem realiza integração real entre navegador e servidor; essa evolução será feita no quarto bimestre.

## 3 CONCEPÇÃO DO SISTEMA E INTEGRAÇÃO ENTRE DISCIPLINAS

Nós organizamos o projeto como um sistema interdisciplinar. Design define como o usuário compreende estados e ações; JavaScript transforma a interface em uma experiência dinâmica; Flask centraliza regras e comunicação; Banco de Dados garante persistência e relacionamentos; Segurança controla identidade, permissão e validação. Nenhuma dessas partes resolve isoladamente o problema: a qualidade do fluxo depende da concordância entre todas elas.

| Disciplina     | Contribuição realizada                                                                                      | Evolução arquitetural                                                                    |
|----------------|-------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Design         | Paleta, tipografia, componentes, telas por perfil, Cardápio e KDS.                                          | Fidelidade ao protótipo, acessibilidade e testes de usabilidade.                         |
| JavaScript     | MPA navegável, módulos JS, arrays/objetos, loops modernos, DOM, validações, carrinho e cálculos em memória. | Classes ES6+, módulos de API, Fetch/JSON, async/await e integração com Flask.            |
| Python e Flask | Estrutura Flask inicial e endpoints simulados para autenticação, Reservas, Cardápio, Pedidos e KDS.         | MVC completo, services/models reais, banco persistente, autenticação e rotas protegidas. |
| Banco de Dados | DER, MER, EER e dicionário de dados.                                                                        | Modelo físico, scripts SQL, consultas e triggers.                                        |
| Segurança      | Definição de perfis e separação visual.                                                                     | Autenticação, sessão, RBAC no servidor e sanitização.                                    |

## 4 ATORES E RESPONSABILIDADES

| Ator          | Objetivo no sistema                                 | Principais ações                                                                          |
|---------------|-----------------------------------------------------|-------------------------------------------------------------------------------------------|
| Cliente       | Utilizar uma Bancada e consumir durante uma Sessão. | Consultar Bancadas, reservar, abrir Sessão, montar carrinho, enviar e acompanhar Pedidos. |
| Cozinheiro    | Organizar a produção.                               | Consultar fila, iniciar preparo, marcar Pedido pronto e entregue, acompanhar tempos.      |
| Administrador | Configurar e supervisionar a operação.              | Gerenciar Bancadas, Reservas, Pratos, Ingredientes, usuários e relatórios.                |

Nós representamos os três atores por menus distintos. Essa separação melhora a compreensão do protótipo, mas sabemos que visibilidade de menu não equivale a autorização. Na aplicação integrada, cada requisição protegida precisará ser validada pelo Flask de acordo com o usuário autenticado e seu papel.

## 5 FLUXO GERAL DO PRODUTO

O fluxo começa quando o Cliente consulta uma data e um intervalo. O sistema compara capacidade e disponibilidade das Bancadas. Com uma opção válida, cria a Reserva e preserva o período e o valor acordado. No início do atendimento, a Reserva gera uma Sessão única. É a Sessão ativa que autoriza o acesso ao Cardápio e a criação de Pedidos. A cozinha recebe o Pedido no KDS, altera seu estado e devolve esse progresso ao Cliente. No encerramento, a Sessão deixa de aceitar novos Pedidos e conserva o histórico.

| Etapa    | Estado de entrada   | Processamento                                    | Saída                                 |
|----------|---------------------|--------------------------------------------------|---------------------------------------|
| Reserva  | Bancada disponível. | Validamos período, capacidade e conflito.        | Reserva confirmada.                   |
| Sessão   | Reserva válida.     | Criamos contexto único e ativamos o atendimento. | Sessão ativa.                         |
| Cardápio | Sessão ativa.       | Exibimos Pratos disponíveis e mantemos carrinho. | Itens selecionados.                   |
| Pedido   | Carrinho válido.    | Revalidamos itens, preços e quantidades.         | Pedido recebido.                      |
| KDS      | Pedido recebido.    | Organizamos fila e controlamos transições.       | Pedido em preparo, pronto e entregue. |

Esse fluxo também orienta a estrutura de dados. O Pedido não precisa repetir Cliente ou Bancada, porque sua Sessão conduz à Reserva e a Reserva identifica ambos. Essa decisão reduz redundância e mantém uma única fonte para o contexto do atendimento.

## 6 RESERVA

### 6.1 Lógica de negócio

Nós tratamos a Reserva como o compromisso de uso de uma Bancada durante um intervalo. Para criar uma Reserva, precisamos identificar o Cliente, a Bancada, o início, o fim, o número de pessoas e o valor. A Bancada deve estar ativa, comportar o grupo e não possuir outra Reserva incompatível. O fim precisa ser posterior ao início.

A condição abstrata de conflito é: há sobreposição quando o início solicitado ocorre antes do fim existente e o fim solicitado ocorre depois do início existente. Essa regra deve ser executada no servidor imediatamente antes da gravação. A tela pode antecipar a disponibilidade, mas somente o back-end pode confirmar, pois outra pessoa pode reservar o mesmo intervalo enquanto o primeiro Cliente ainda preenche o formulário.

### 6.2 Contribuição de cada disciplina

Em Design, criamos cartões de Bancada, etiquetas de disponibilidade, filtros e formulário de Reserva. Em JavaScript, utilizamos o array benches e templates para renderizar capacidade, recursos, preço e estado. Em Banco de Dados, relacionamos CLIENTE e BANCADA à RESERVA. Em Flask, definimos a necessidade de rotas de consulta, criação, alteração e cancelamento. Em Segurança, estabelecemos que o Cliente só pode manipular suas próprias Reservas, enquanto o Administrador possui acesso operacional mais amplo.

```
const benches = [  
  {id:1, nome:"Bancada Toscana", cap:4, preco:85,  
    status:"Disponível", recursos:["Cooktop","Pia","Tomadas"]}  
];
```

*Trecho 1 - Estrutura real utilizada para representar Bancadas no protótipo.*

No protótipo, os dados permanecem em memória e os botões simulam confirmação. Na aplicação final, o JavaScript enviará os campos em JSON, exibirá estado de carregamento e tratará respostas 201, 400 ou 409 sem recarregar a página.

## 7 SESSÃO

Nós adotamos Sessão como entidade central do atendimento. Uma Reserva descreve o período contratado; a Sessão registra o uso efetivo e reúne os Pedidos daquele período. Cada Reserva deve gerar exatamente uma Sessão, protegida por uma relação um para um. A Sessão possui código de acesso, estado e datas de criação, início e encerramento.

Uma Sessão pode estar criada, ativa ou encerrada. Somente ativa aceita Pedidos. A ativação registra o início real; o encerramento invalida o código de acesso e bloqueia novas solicitações. Essa verificação precisa acontecer no Flask a cada Pedido, pois não podemos confiar apenas no botão da interface.

Na tela que desenvolvemos, mostramos tempo restante, quantidade de Pedidos, consumo, pessoas, Pedidos vinculados e ações de navegação. Em JavaScript, essa tela é uma função do objeto views. No banco, SESSAO recebe uma referência única à RESERVA e origina vários PEDIDOS. Em Segurança, o acesso deve ser limitado ao Cliente responsável, à operação autorizada e ao Administrador.

## 8 CARDÁPIO E CARRINHO

Nosso Cardápio Digital apresenta foto, nome, descrição, categoria, preço e disponibilidade. Criamos oito produtos demonstrativos e organizamos a experiência em Entradas, Pratos, Bebidas e Sobremesas. Quando um item está indisponível, desabilitamos seu botão de adição. Essa regra visual ajuda o usuário, mas o servidor deverá revalidá-la na confirmação.

### 8.1 Lógica do carrinho

```
function addCart(id){
  const p=products.find(x=>x.id===id);
  if(p.status!=="Disponível") return;
  const ex=state.cart.find(x=>x.id===id);
  if(ex) ex.qtd++;
  else state.cart.push({id:p.id,nome:p.nome,preco:p.preco,qtd:1});
}
function cartTotal(){
  return state.cart.reduce((s,i)=>s+i.preco*i.qtd,0);
}
```

*Trecho 2 - Adição de item e cálculo do total no protótipo.*

Primeiro localizamos o produto pelo identificador. Se ele estiver indisponível, encerramos a função. Depois procuramos o mesmo item no carrinho: quando existe,

incrementamos a quantidade; quando não existe, adicionamos um novo objeto. O total é calculado com `reduce`, acumulando preço multiplicado pela quantidade. Essa lógica satisfaz o objetivo do terceiro bimestre de manipular arrays, objetos e cálculos em memória.

Na integração, o navegador enviará identificadores e quantidades, mas não será responsável pelo valor final. O Flask consultará os preços atuais, validará cada Prato e calculará os subtotais. Isso evita que um preço alterado no console do navegador seja aceito como verdadeiro.

## 9 PEDIDOS

O Pedido agrupa itens solicitados durante uma Sessão. Definimos que ele deve possuir ao menos um `ITEM_PEDIDO`, estado, valor total, tempo estimado e data de criação. `ITEM_PEDIDO` preserva quantidade, preço unitário praticado e subtotal. Mesmo que o preço do catálogo mude posteriormente, o histórico continua representando a compra original.

Ao confirmar, o servidor deverá verificar autenticação, propriedade da Sessão, estado ativo, lista não vazia, quantidades positivas e Pratos disponíveis. Em seguida, buscará os preços, calculará subtotais, gravará Pedido e itens na mesma transação e devolverá JSON. Se qualquer etapa falhar, a transação inteira deve ser desfeita.

| Estado                 | Significado                             | Próxima ação esperada                            |
|------------------------|-----------------------------------------|--------------------------------------------------|
| AGUARDANDO ou RECEBIDO | Pedido aceito e visível para a cozinha. | Iniciar preparo ou cancelar conforme regra.      |
| EM PREPARO             | Produção iniciada.                      | Marcar como pronto.                              |
| PRONTO                 | Pedido finalizado pela cozinha.         | Marcar como entregue.                            |
| ENTREGUE               | Ciclo operacional concluído.            | Somente consulta histórica.                      |
| CANCELADO              | Pedido interrompido.                    | Somente consulta ou procedimento administrativo. |

## 10 KDS

O Kitchen Display System é o painel do Cozinheiro. Nós o organizamos em colunas para tornar o estado imediatamente visível: Recebidos, Em preparo e Prontos. Cada ticket apresenta número, Bancada, tempo e itens. Cores e etiquetas reforçam prioridade e atraso. Essa estrutura reduz a necessidade de abrir cada Pedido separadamente.



No protótipo, os tickets são exemplos fixos e os botões acionam um toast. Na arquitetura definida, o KDS consultará a API e enviará alterações de estado. A interface só moverá o ticket depois da confirmação do servidor. O Flask validará a transição para impedir saltos ou regressões incoerentes. Dois Cozinheiros atuando ao mesmo tempo deverão receber uma resposta de conflito quando tentarem atualizar uma versão antiga do Pedido.

Design contribui com hierarquia, cores e leitura rápida. JavaScript controla renderização e atualização sem F5. Flask organiza fila e transições. Banco registra estado e horários. Segurança limita as ações a Cozinheiro e Administrador. Assim, o KDS reúne todas as disciplinas em uma única função operacional.

## 11 ADMINISTRAÇÃO DE BANCADAS E CATÁLOGO

O Administrador configura os recursos utilizados nos fluxos anteriores. Desenvolvemos telas para Bancadas, Reservas, Produtos, Ingredientes, substituições, usuários e relatórios. As operações exibidas ainda são demonstrativas, mas definem a experiência e os dados necessários para os futuros CRUDs.

A exclusão de Prato deve ser lógica por meio do atributo ativo. Dessa forma, um item deixa de aparecer para novas compras sem destruir ITEM\_PEDIDO histórico. A ficha técnica relaciona Prato e Ingrediente com quantidade necessária. A substituição é direcional e possui prioridade e fator de conversão. A futura disponibilidade poderá somar o consumo de todos os itens e escolher substitutos autorizados quando o ingrediente original estiver insuficiente.

## 12 ARQUITETURA DA APLICAÇÃO

Nós definimos uma arquitetura em camadas. O navegador contém HTML, CSS e JavaScript; a comunicação utiliza HTTP e JSON; o Flask recebe requisições, autentica, valida e executa regras; a camada de dados persiste entidades relacionais. O banco nunca deve ser acessado diretamente pelo navegador.

| Camada               | Responsabilidade                                    | Exemplo                       |
|----------------------|-----------------------------------------------------|-------------------------------|
| View                 | Apresentar conteúdo e capturar eventos.             | Cardápio, formulário e KDS.   |
| JavaScript de página | Gerenciar estado visual e chamar API.               | Carrinho, filtros e feedback. |
| Routes Flask         | Interpretar HTTP, parâmetros e códigos de resposta. | POST /api/reservas.           |

| Camada                 | Responsabilidade                     | Exemplo                            |
|------------------------|--------------------------------------|------------------------------------|
| Controllers e services | Executar caso de uso e regras.       | Verificar conflito e criar Sessão. |
| Models e repositórios  | Representar e consultar dados.       | Reserva, Sessão, Pedido e Prato.   |
| Banco relacional       | Garantir persistência e integridade. | PK, FK, unicidade e transações.    |

O projeto adota separação por responsabilidades. No front-end, a lógica foi dividida em arquivos específicos para dados, navegação, componentes, cardápio, carrinho, validação, login, views e acessibilidade. No back-end simulado, `run.py` inicializa o Flask, `config.py` concentra configurações e os controllers exemplificam autenticação, reservas, cardápio, pedidos e KDS. A estrutura completa com routes, services, models e middleware permanece como evolução arquitetural.

## 13 LÓGICA JAVASCRIPT

O JavaScript atual foi dividido por responsabilidade. `data.js` concentra arrays, objetos literais e estado compartilhado; `site.js` reúne navegação e helpers; `components.js` gera componentes visuais; `catalog.js` manipula dinamicamente o DOM do cardápio; `cart.js` mantém o carrinho e calcula totais; `validation.js` valida a reserva; `login.js` trata o login demonstrativo; `views.js` registra as telas; e `a11y.js` concentra comportamentos de acessibilidade. A navegação ocorre entre páginas HTML separadas, caracterizando uma MPA.

```
function renderCatalog(category = "Todos") {
  const grid = document.querySelector(".grid.cols-4");
  grid.innerHTML = "";
  products.forEach(product => {
    if (category !== "Todos" && product.cat !== category) return;
    const wrapper = document.createElement("div");
    wrapper.innerHTML = productCard(product).trim();
    grid.appendChild(wrapper.firstElementChild);
  });
}
```

Trecho 3 - Renderização dinâmica do Cardápio utilizando `forEach` e manipulação do DOM.

Utilizamos arrays, objetos literais, `const`, arrow functions, template literals, `find`, `filter`, `reduce`, `forEach`, `for...of` e `for...in`. O Cardápio é atualizado dinamicamente pelo DOM; o carrinho mantém itens e calcula subtotais e total em memória; os formulários possuem validações simples; e o login demonstrativo seleciona o perfil e armazena a sessão localmente. Esses elementos cobrem os conteúdos de JavaScript previstos para o terceiro bimestre.

No quarto bimestre, a modularização será ampliada com classes ES6+, encapsulamento e módulos voltados à comunicação com a API. As chamadas Fetch serão assíncronas, utilizarão JSON e serão tratadas com `async/await` e `try/catch`. O estado que hoje é simulado no navegador passará a refletir as respostas reais do Flask.

## 14 ARQUITETURA FLASK E ROTAS ESSENCIAIS

O back-end do terceiro bimestre é propositalmente simulado. Ele já possui `run.py`, `config.py`, `app/__init__.py` e controllers de autenticação, reserva e pedidos. Esses arquivos demonstram os endpoints fundamentais e a separação de responsabilidades, mas os models ainda são stubs e a API não está integrada ao banco ou ao front-end. O objetivo desta etapa é validar o contrato das operações antes da implementação definitiva.

| Método e rota                  | Atores           | Lógica essencial                                                                    |
|--------------------------------|------------------|-------------------------------------------------------------------------------------|
| POST /api/auth/login           | Todos            | Validar credenciais, usuário ativo, criar Sessão de autenticação e retornar perfil. |
| GET /api/bancadas              | Cliente/Admin    | Filtrar capacidade e ausência de conflito no intervalo.                             |
| POST /api/reservas             | Cliente/Admin    | Revalidar disponibilidade em transação, calcular valor e criar Reserva.             |
| POST /api/reservas/{id}/sessao | Cliente/Admin    | Garantir Reserva válida e Sessão única; gerar código de acesso.                     |
| GET /api/cardapio              | Cliente          | Listar Pratos ativos e disponíveis.                                                 |
| POST /api/sesoes/{id}/pedidos  | Cliente          | Validar Sessão e itens; recalcular preços; gravar Pedido e itens.                   |
| GET /api/kds/pedidos           | Cozinheiro/Admin | Listar fila por estado e tempo.                                                     |
| PATCH /api/pedidos/{id}/status | Cozinheiro/Admin | Validar transição, atualizar estado e horários.                                     |
| POST /api/sesoes/{id}/encerrar | Cliente/Admin    | Bloquear novos Pedidos e registrar encerramento.                                    |

### 14.1 Exemplo teórico de rota

```
# .../api/reservas - método POST
def post_reserva(dados):
    cliente = dados["cliente"]
    bancada = dados["bancada"]
    Reserva_model.post_reserva(cliente, bancada)
    return True, 200
```

Trecho 4 - Exemplo atual do back-end simulado para criação de Reserva.

O exemplo atual demonstra o contrato básico de uma operação: receber dados, encaminhá-los para a camada responsável e devolver resultado e código HTTP. Nesta

primeira entrega, os models são simulados. Na implementação definitiva, controllers e services aplicarão autenticação, validação, regras de negócio e transações antes de acessar o banco.

Utilizaremos 200 para leituras e alterações bem-sucedidas, 201 para criação, 400 para dados inválidos, 401 para falta de autenticação, 403 para papel inadequado, 404 para recurso inexistente e 409 para conflito de Reserva, Sessão ou estado. O JavaScript interpretará esses códigos e mostrará feedback específico.

## 15 BANCO DE DADOS E DER

Nós modelamos USUARIO e suas especializações CLIENTE, COZINHEIRO e ADMINISTRADOR; BANCADA e RESERVA; SESSAO, PEDIDO e ITEM\_PEDIDO; PRATO e INGREDIENTE; além de FICHA\_TECNICA e SUBSTITUIDO\_POR. O dicionário descreve atributos, tipos sugeridos, domínios e regras de consistência.

*Figura 1 - Diagrama Entidade Relacionamento do Sabor e Clic. Fonte: elaboração da equipe, 2026.*

A cadeia RESERVA ↔ SESSAO → PEDIDO → ITEM\_PEDIDO é o núcleo do modelo. CLIENTE solicita RESERVA e BANCADA é utilizada pela RESERVA. Cada item referencia um PRATO. FICHA\_TECNICA resolve a relação muitos para muitos entre Prato e Ingrediente, enquanto SUBSTITUIDO\_POR utiliza duas referências para Ingrediente.

*Figura 2 - Modelo de entidades e relacionamentos. Fonte: elaboração da equipe, 2026.*

Definimos regras de integridade: e-mail único, valores não negativos, fim posterior ao início, Sessão única por Reserva, quantidade positiva, Pedido com itens e chaves estrangeiras obrigatórias. No quarto bimestre, o modelo físico deverá traduzir essas regras para SQL, constraints, índices, inserções básicas, consultas e triggers de estoque.

## 16 SEGURANÇA E NÍVEIS DE ACESSO

Definimos RBAC com três papéis. Cliente acessa seus próprios dados; Cozinheiro opera Pedidos e KDS; Administrador gerencia cadastros e visão geral. No protótipo, essa

divisão aparece nos menus. Na aplicação final, ela será aplicada no servidor por decorators ou middleware, porque ocultar um botão não impede uma chamada HTTP direta.

| Risco             | Decisão arquitetural                                                                                             |
|-------------------|------------------------------------------------------------------------------------------------------------------|
| Senha exposta     | Armazenar hash forte com salt; nunca senha em texto ou cifra reversível.                                         |
| Preço manipulado  | Recalcular total no Flask a partir do catálogo.                                                                  |
| Reserva duplicada | Validar sobreposição dentro de transação.                                                                        |
| SQL Injection     | Utilizar ORM ou consultas parametrizadas.                                                                        |
| XSS               | Escapar dados e preferir <code>textContent</code> para conteúdo externo.                                         |
| CSRF/cookie       | Configurar <code>HttpOnly</code> , <code>Secure</code> , <code>SameSite</code> e proteção CSRF quando aplicável. |
| Acesso indevido   | Verificar identidade, propriedade do recurso e papel em cada rota.                                               |

O utilitário Fernet existente pode servir para dados que precisem ser recuperados, mas não para senha. Para senha, adotaremos hash unidirecional. Também planejamos logs sem informações sensíveis, mensagens genéricas para falhas de login e validação duplicada no cliente e no servidor.

## 17 IDENTIDADE VISUAL E PROTOTIPAÇÃO

Nós construímos uma identidade inspirada em materiais e cores gastronômicas. Utilizamos grafite #2B241E para estrutura, fundo #F6F1E8, cobre #BB6B2F como destaque, verde #4B7A52 para sucesso, açafrão #B9832A para atenção e páprica #A6432E para erro. Archivo aparece em títulos e Hanken Grotesk nos elementos de interface.

Criamos componentes consistentes: menu lateral, cartões, badges, botões, tabelas, filtros, indicadores, timeline e colunas do KDS. CSS Grid e Flexbox organizam o conteúdo, enquanto media queries adaptam a interface. As imagens das Bancadas e dos produtos aproximam o protótipo de uma experiência de uso real.

Ainda precisamos incorporar ao repositório os wireframes e o link/exportação do protótipo Figma. O front-end atual funciona como protótipo navegável codificado e demonstra telas de Cliente, Cozinheiro e Administrador.

[INSERIR FIGURA - WIREFRAMES E PROTÓTIPO FIGMA]

## 18 REQUISITOS FUNCIONAIS E NÃO FUNCIONAIS

| Código | Requisito funcional                                 | Estágio                                                      |
|--------|-----------------------------------------------------|--------------------------------------------------------------|
| RF01   | Entrar e acessar interface correspondente ao papel. | Demonstrado no protótipo; autenticação real é próxima etapa. |
| RF02   | Consultar e reservar Bancadas.                      | Interface concluída; persistência planejada.                 |
| RF03   | Criar, ativar e encerrar Sessão.                    | Modelado e prototipado.                                      |
| RF04   | Consultar Cardápio e manter carrinho.               | Implementado em memória.                                     |
| RF05   | Criar e acompanhar Pedido.                          | Interface concluída; API planejada.                          |
| RF06   | Operar fila no KDS.                                 | Layout concluído; atualização persistente planejada.         |
| RF07   | Administrar catálogo, Ingredientes e usuários.      | Telas demonstrativas concluídas; CRUD planejado.             |

| Requisito não funcional | Como tratamos                                                      |
|-------------------------|--------------------------------------------------------------------|
| Usabilidade             | Hierarquia, feedback, navegação por perfil e consistência visual.  |
| Responsividade          | Grid, Flexbox e media queries no CSS.                              |
| Manutenibilidade        | Planejamento modular de pages, models, api, routes e services.     |
| Segurança               | RBAC, validação no servidor, hash e consultas parametrizadas.      |
| Integridade             | Transações, PK/FK, unicidade e domínios de estado.                 |
| Desempenho              | Filtros no servidor, índices futuros e atualização parcial do DOM. |

## 19 TECNOLOGIAS UTILIZADAS

| Tecnologia                       | Aplicação no projeto                                                        |
|----------------------------------|-----------------------------------------------------------------------------|
| HTML5                            | Estrutura das páginas e formulários.                                        |
| CSS3                             | Identidade visual, componentes e responsividade.                            |
| JavaScript ES6+                  | DOM, estado, arrays, templates, carrinho e cálculos.                        |
| JSON e Fetch API                 | Formato e mecanismo definidos para integração; exemplo inicial no back-end. |
| Python e Flask                   | Base escolhida para API, regras e autenticação.                             |
| Banco relacional/MySQL Workbench | DER, MER, EER e futura persistência SQL.                                    |
| Git e GitHub                     | Versionamento e integração do trabalho da equipe.                           |

## 20 O QUE CONCLUÍMOS E PRÓXIMAS ETAPAS

### 20.1 O que concluímos

Concluímos o protótipo navegável, a identidade visual, as telas dos três atores, o Cardápio, o carrinho em memória, a visualização de Pedidos, o layout do KDS, o DER, o

MER, o arquivo EER e o dicionário de dados. Também modularizamos o JavaScript para evidenciar arrays, objetos, for...of, for...in, forEach, DOM, validações e cálculos em memória, e estruturamos um back-end Flask simulado com endpoints fundamentais.

## 20.2 O que iniciamos

Implementamos uma primeira estrutura Flask com configuração da aplicação e controllers simulados para cadastro/login, Bancadas/Reservas e Cardápio/Pedidos/KDS. Esses endpoints ainda não formam uma API persistente, mas já estabelecem os contratos que serão ligados ao front-end. A autenticação real, o banco físico e Fetch/JSON continuam como próxima etapa.

## 20.3 Próximas etapas

Nas próximas etapas, completaremos a factory Flask e a separação MVC, substituiremos os models simulados por persistência real, implementaremos autenticação e RBAC, criaremos a Sessão persistente, integraremos Cardápio/Pedido/KDS ao banco e conectaremos o front-end à API por Fetch/JSON. Depois serão acrescentados tratamento de loading/erros e testes para conflitos, permissões, totais e transições de estado.

## 21 ATENDIMENTO AOS REQUISITOS DO TERCEIRO BIMESTRE

| Requisito            | Situação                           | Evidência produzida                                                       |
|----------------------|------------------------------------|---------------------------------------------------------------------------|
| DER                  | Concluído                          | DER_saborcllc.png, MER_saborcllc.png e EER_saborcllc.mwb.                 |
| Dicionário de dados  | Concluído                          | Documento em DOCX e PDF com entidades, atributos e relações.              |
| Wireframes/Figma     | Em complementação                  | Protótipo codificado existe; arquivo ou link Figma deverá ser incluído.   |
| Identidade visual    | Concluída                          | Paleta, tipografia, marca e componentes em CSS.                           |
| Cardápio Digital     | Concluído como protótipo           | Produtos, categorias, imagens, disponibilidade e carrinho.                |
| KDS                  | Concluído como protótipo           | Colunas, tickets, tempos e ações simuladas.                               |
| JavaScript/DOM       | Concluído para a etapa             | Renderização dinâmica, eventos, arrays, map/filter/reduce e cálculo.      |
| Validações           | Parcial                            | required e regras locais; validações completas serão repetidas no Flask.  |
| Flask/MVC/rotas/JSON | Estrutura teórica e início técnico | Arquivos iniciais, exemplo fetch e arquitetura detalhada neste documento. |
| Segurança e RBAC     | Definido conceitualmente           | Papéis e menus; middleware e autenticação serão implementados.            |

## REFERÊNCIAS

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO. Projeto Integrador Sabor & Clic: sistema de gestão para dark kitchens e espaços gastronômicos. Versão de 31 de agosto de 2026. São Paulo: IFSP, 2026.

SABOR E CLIC. Repositório Sabor-Clic. GitHub, 2026. Disponível em: <https://github.com/FaellEmiliano/Sabor-Clic>.

SABOR E CLIC. Dicionário de Dados Sabor e Clic. São Paulo, 2026.



## 22 ESTADO ATUAL DA IMPLEMENTAÇÃO E INTEGRAÇÃO

Esta seção consolida o estado do repositório após a evolução do terceiro bimestre. Ela complementa os capítulos anteriores sem substituir a modelagem, as figuras, os protótipos e as decisões já documentadas.

### 22.1 Páginas atuais do front-end

- Acesso: index.html.
- Cliente: cliente\_inicio.html, cliente\_bancadas.html, cliente\_reservas.html, cliente\_sessao.html, cliente\_cardapio.html, cliente\_carrinho.html e cliente\_pedidos.html.
- Cozinha: cozinha\_dashboard.html, cozinha\_kds.html, cozinha\_pedidos.html, cozinha\_estoque.html e cozinha\_historico.html.
- Administrador: admin\_dashboard.html, admin\_reservas.html, admin\_bancadas.html, admin\_produtos.html, admin\_ingredientes.html, admin\_substituicoes.html, admin\_usuarios.html e admin\_relatorios.html.

### 22.2 Módulos JavaScript atuais

| Arquivo       | Responsabilidade                  | Requisito demonstrado                 |
|---------------|-----------------------------------|---------------------------------------|
| data.js       | Dados simulados e estado          | arrays, objetos, for...in             |
| site.js       | Navegação, helpers e sessão local | DOM, eventos e localStorage           |
| components.js | Componentes reutilizáveis         | for...of e templates                  |
| catalog.js    | Cardápio dinâmico                 | forEach e manipulação do DOM          |
| cart.js       | Carrinho e totais                 | find/filter/reduce/forEach e cálculos |
| validation.js | Validação da Reserva              | for...of e validação                  |
| login.js      | Login demonstrativo               | validação e perfis                    |

|          |                       |                           |
|----------|-----------------------|---------------------------|
| views.js | Templates das páginas | renderização do protótipo |
| a11y.js  | Acessibilidade        | comportamentos auxiliares |

### 22.3 Endpoints do back-end simulado

| Método | Endpoint             | Finalidade                       | Estado   |
|--------|----------------------|----------------------------------|----------|
| POST   | /api/cadastro        | Cadastrar cliente                | Simulado |
| GET    | /api/cadastro/<id>   | Consultar usuário                | Simulado |
| POST   | /api/login           | Validar login                    | Simulado |
| GET    | /api/bancadas        | Listar Bancadas                  | Simulado |
| POST   | /api/reservas        | Criar Reserva                    | Simulado |
| GET    | /api/reservas/<id>   | Consultar Reserva                | Simulado |
| GET    | /api/cardapio        | Listar Cardápio                  | Simulado |
| POST   | /api/pedido/cadastro | Criar Pedido vinculado à Reserva | Simulado |
| GET    | /api/pedido/<id>     | Consultar Pedido                 | Simulado |
| GET    | /api/kds/<id>        | Gerar consulta para o KDS        | Simulado |

### 22.4 Integração conceitual entre os sistemas

1. O Cliente seleciona uma Bancada e envia os dados da Reserva.
2. A Reserva cria o contexto que dará origem à Sessão.

3. A Sessão ativa permitirá acesso ao Cardápio e criação de Pedidos.
4. O JavaScript mantém o carrinho e apresenta os cálculos ao usuário.
5. No quarto bimestre, o navegador enviará apenas identificadores e quantidades; o Flask recalculará preços e persistirá Pedido e itens.
6. O KDS consumirá os Pedidos persistidos e enviará alterações de estado ao servidor.

## 22.5 Limite da simulação e evolução para o quarto bimestre

A filosofia adotada nesta entrega é demonstrar os conceitos estudados com exemplos simples e coerentes com a arquitetura final. O front-end usa mocks e localStorage; o Flask usa controllers e models simulados. O objetivo não é antecipar o sistema completo, mas evitar código descartável: os mesmos conceitos de Reserva, Sessão, Cardápio, Pedido e KDS serão mantidos quando a persistência, Fetch/JSON, autenticação e regras completas forem implementados.

## 22.6 Situação dos requisitos principais do terceiro bimestre

| Requisito                     | Situação                           | Evidência atual                                    |
|-------------------------------|------------------------------------|----------------------------------------------------|
| DER e dicionário de dados     | Concluído                          | Arquivos e figuras já incorporados à documentação. |
| Protótipo navegável           | Concluído                          | Páginas de Cliente, Cozinha e Administrador.       |
| Cardápio Digital              | Concluído como protótipo funcional | cliente_cardapio.html + catalog.js.                |
| KDS                           | Concluído como protótipo visual    | cozinha_kds.html + views.js.                       |
| Arrays e objetos              | Concluído                          | data.js.                                           |
| for...of / for...in / forEach | Concluído                          | components.js, data.js, catalog.js e cart.js.      |
| DOM dinâmico                  | Concluído                          | catalog.js e scripts de interface.                 |

|                          |                            |                                  |
|--------------------------|----------------------------|----------------------------------|
| Validação de formulários | Concluído em nível inicial | validation.js e login.js.        |
| Cálculos em memória      | Concluído                  | cart.js.                         |
| Estrutura Flask          | Concluída em nível inicial | run.py, config.py e app.         |
| Endpoints fundamentais   | Concluídos como simulação  | controllers Flask.               |
| Fetch/JSON integrado     | Próxima etapa              | Previsto para o quarto bimestre. |